

项目描述

在上一个 SFT (Supervised Fine-Tuning) 推荐模型微调项目里，介绍了推荐系统的概念和大模型在推荐系统方向中的应用方法，并以 MIND 数据集为例进行了新闻推荐模型有监督微调。

SFT 训练方法（比如 TALLRec [1]）在训练时只考虑了“正样本”的特征，在对未知样本预测时可能会产生“过度”推断的问题。推荐问题的本质是一个排序 Rank，因此在训练阶段就需要建模涉及排序的“正样本”和“负样本”。RLHF (Reinforcement Learning from Human Feedback) 训练方法构建了 chosen 和 rejected 成对数据进行如 PPO (Proximal Policy Optimization) 或 DPO (Direct Preference Optimization) [2] 或 ORPO (Odds Ratio Preference Optimization) [3] 的训练。而推荐问题的目标是预测一个排序，因此在训练时也要将排序考虑在内。借鉴对比学习的方法，对于一个“正样本”，DMPO (Direct Multi-Preference Optimization) 方法[4]在训练时加入了多个“负样本”，比传统 RLHF 应用在推荐系统中效果更好。

在本项目中，以 Amazon Video Games 数据集 [5] 为例，讲述了数据集 RLHF 预处理方法，同时基于 GLM4 [6] 的进行 RLHF 训练和训练后的模型预测。整个项目文件列表如下：

-  第二步：模型预测.ipynb
-  第一步：数据预处理.ipynb
- >  amazon_video_games_orpo
- >  data
- >  materials
-  rlhf_inference.yaml
-  rlhf_train.yaml

其中 , amazon_video_games_orpo 为训练好的 LoRA 参数文件 , 其余文件在如下文档中皆有介绍。

项目目标

- 熟悉 RLHF 在推荐系统中应用的一般方式
- 掌握 RLHF 在推荐系统中的数据构建和模型训练

RLHF 在推荐系统中应用的一般方式

DPO , ORPO 和 DMPO

传统 RLHF 训练包含 Reward 模型和 PPO 对齐两个训练阶段。PPO 是一种强化学习训练方法 , 训练时比较困难。DPO 避免了强化学习的训练 , 直接优化最优策略 π_θ , 具体如下 :

$$\mathcal{L}_{\text{DPO}}(\pi_\theta; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_\theta(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_\theta(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right].$$

其中 , π_{ref} 是参考策略。

ORPO 方法则更进一步 , 不需要参考策略 , 基于 odds ratio , 同时 ORPO 方法的损失函数也加入 SFT 损失 , 可以直接基于 Pre-trained 模型进行 RLHF 训练 , 略过 SFT 阶段的训练。

$$\mathcal{L}_{OR} = -\log \sigma \left(\log \frac{\text{odds}_\theta(y_w | x)}{\text{odds}_\theta(y_l | x)} \right)$$

$$\mathcal{L}_{ORPO} = \mathbb{E}_{(x, y_w, y_l)} [\mathcal{L}_{\text{SFT}} + \lambda \cdot \mathcal{L}_{OR}]$$

DMPO 是 DPO 的优化版本，其优化目标如下所示：

$$\mathcal{L}_{\text{DMPO}}(\pi_{\theta}; \pi_{\text{ref}}) = -\mathbb{E}_{(x, y_w, y_l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \frac{1}{k} \sum_{i=1}^k (\beta \log \frac{\pi_{\theta}(y_{l_i} | x_i)}{\pi_{\text{ref}}(y_{l_i} | x_i)}) \right) \right]$$

其中， k 表示负样本的个数。与 DPO 的目标函数相比，DPO 只考虑了一个负样本，而 DMPO 考虑了 k 个负样本。

RLHF 在推荐系统中的数据构建和模型训练

数据

在本项目中，我们使用 Amazon Video Games 数据集 [5]。Amazon 数据集包括了 1996 年 5 月到 2014 年 7 月期间 142.8 百万的用户商品评价信息，包括了 Books, Movies and TV, Video Games, Electronics 等子类别，具体信息包括评价用户 ID，商品 ID，商品标题，商品品牌，用户-商品交互，评价等级和具体评价内容等。整体来说，不同子类别包含的文件类型是一致的，即商品信息和用户-商品交互信息，商品信息字段如下：

- asin - ID of the product, e.g. [0000031852](#)
- title - name of the product
- feature - bullet-point format features of the product
- description - description of the product
- price - price in US dollars (at time of crawl)
- imageURL - url of the product image
- imageURL - url of the high resolution product image
- related - related products (also bought, also viewed, bought together, buy after viewing)
- salesRank - sales rank information
- brand - brand name
- categories - list of categories the product belongs to
- tech1 - the first technical detail table of the product
- tech2 - the second technical detail table of the product
- similar - similar product table

用户-商品交互信息字段如下：

- reviewerID - ID of the reviewer, e.g. [A2SUAM1J3GNN3B](#)
- asin - ID of the product, e.g. [0000013714](#)
- reviewerName - name of the reviewer
- vote - helpful votes of the review
- style - a dictionary of the product metadata, e.g., "Format" is "Hardcover"
- reviewText - text of the review
- overall - rating of the product
- summary - summary of the review
- unixReviewTime - time of the review (unix time)
- reviewTime - time of the review (raw)
- image - images that users post after they have received the product

我们选择 Video Games 子类别进行 RLHF 数据集构建。Amazon Video Games 原始数据集目录为 data/src_data，包括 2 个文件，meta_Video_Games.json 文件包括了 Video Games 的具体 ID，名称等属性信息，示例样本如下：

```
{
  "category": ["Video Games", "PC", "Games"],
  "tech1": "",
  "description": [],
  "fit": "",
  "title": "Reversi Sensory Challenger",
  "also_buy": [],
  "tech2": "",
  "brand": "Fidelity Electronics",
  "feature": [],
  "rank": [">#2,623,937 in Toys & Games (See Top 100 in Toys & Games)"],
  ">#39,015 in Video Games & PC Games"],
  "also_view": [],
  "main_cat": "Toys & Games",
  "similar_item": "",
  "date": "",
  "price": "",
  "asin": "0042000742",
  "imageURL": ["https://images-na.ssl-images-amazon.com/images/I/31nTxLNh10L._SS40_.jpg"],
  "imageURLHighRes": ["https://images-na.ssl-images-amazon.com/images/I/31nTxLNh10L.jpg"]}

```

Video_Games_5.json 文件包括了用户对 Video Games 的评论内容和评分等详细信息，示例样本如下：

```
{
  "overall": 5.0,
  "verified": true,
  "reviewTime": "10 17, 2015",
  "reviewerID": "A1HP7NVNPFMA4N",
  "asin": "0700026657",
  "reviewerName": "Ambrosia075",
  "reviewText": "This game is a bit hard to get the hang of, but when you do it's great.",
  "summary": "but when you do it's great.",
  "unixReviewTime": 1445040000
}

```

数据预处理

基于上面的 Amazon Video Games 数据集，我们采用 第一步：数据预处理.ipynb 文件进行数据预处理，首先处理 meta_Video_Games.json 文件，获取 video game ID 和 title 的映射关系；然后处理 Video_Games_5.json 文件，首先选择评价数大于等于 7 个的样本，对这个样本，根据评价时间从小到大排序，选择最近的 2 个 video game title 作为预测目标。同时将离散的 rating 值转换为 like (rating > 3.0) 和 dislike (rating <= 3.0) 标识。在选择 2 个预

测目标时同样也要求一个为 like，一个为 dislike。同时我们定义了 system prompt。

对处理好的数据随机打乱后 8/2 划分，存储在 data/processed 文件下，训练集为 rlhf_train.json 文件（共 5144 个样本）；测试集为 rlhf_test.json 文件（共 1286 个样本）。

与上一个项目类似，在本项目中，我们继续使用优秀的 LLaMA-Factory 框架 [7] 基于 GLM-4-9B-Chat [6] 开源模型做 Video Games 推荐的 RLHF 微调。

将 rlhf_train.json 放在 LLaMA-Factory 的 data 目录下，然后修改 data 目录下的 dataset_info.json 文件，添加数据集表示为 amazon_video_games，即

```
"amazon_video_games": {  
    "file_name": "rlhf_train.json",  
    "ranking": true,  
    "columns": {  
        "prompt": "instruction",  
        "query": "input",  
        "chosen": "chosen",  
        "rejected": "rejected"  
    }  
}
```

模型微调

使用如下命令进行模型微调 `CUDA_VISIBLE_DEVICES=6 llamafactory-cli train ./rlhf_train.yaml` , 需要注意修改 `yaml` 文件内的 `GLM4` 模型地址 `model_name_or_path`。

模型预测

当模型训练完毕后，我们使用 `第二步：模型预测.ipynb` 文件在 `rlhf_test.json` 文件上预测。具体来说，首先使用如下命令加载模型 `CUDA_VISIBLE_DEVICES=2 llamafactory-cli api rlhf_inference.yaml` , 需要注意修改 `yaml` 文件内的 `GLM4` 模型地址 `model_name_or_path`。`第二步：模型预测.ipynb` 文件在测试集样本上进行预测，在 1286 个样本上得到的准确率为 55.9%。这个分数值不高，但与 [4] 中得到的结果类似。即

Table 2: Performance comparison among traditional sequential recommendation baselines, LLM-based methods, and DMPO. The reported results are presented as the AUC multiplied by 100, with boldface indicating the highest score. ‡: significantly better than all baselines with a t-test $p<0.01$.

Dataset	GCSAN	STAMP	SASRec	GRU4Rec	BERT4Rec	SHAN	CORE	TALLrec	OurMethod
MovieLens 1M	52.77	53.75	54.77	52.75	47.72	51.18	50.32	67.79	70.20‡
Amazon Movie	51.07	53.04	49.05	47.51	50.17	52.88	49.55	57.62	66.60‡
Amazon Game	52.11	51.27	50.23	51.26	50.81	50.35	49.96	53.17	64.51‡

参考文献

[1] TALLRec: An Effective and Efficient Tuning Framework to Align Large Language Model with Recommendation, 2023

[2] Direct Preference Optimization: Your Language Model is Secretly a Reward Model, 2024

[3] ORPO: Monolithic Preference Optimization without Reference Model, 2024

[4] Finetuning Large Language Model for Personalized Ranking, 2024

[5] <https://nijianmo.github.io/amazon/index.html>

[6] <https://github.com/THUDM/GLM-4>

[7] <https://github.com/hiyouga/LLaMA-Factory>